Evidence-Bounded Local Retrieval-Augmented Generation for Game Logs: A Vertical-Slice Case Study in Typed Failure Ownership

Anonymous submission

Abstract

Retrieval-augmented generation (RAG) can expose source evidence, but retrieval alone does not make a generated claim correct, a failure attributable, or an execution path local. We study a deliberately small game-log search vertical slice that treats these properties as independently testable contracts. The system incrementally maps local JSONL records into PostgreSQL/pgvector with CocoIndex, retrieves a frozen evidence set, invokes a local Ollama model with a JSON Schema, and publishes a finding only after deterministic claim–evidence validation. Every terminal state has an explicit owner—retrieval, synthesis, or none—and no alternate RAG stack is a permitted fallback.

We report a case study over nine synthetic records, eight indexed records, ten fixed queries, and one run for each of three Qwen2.5 model profiles (0.5B, 1.5B, and 3B), all recorded as Q4_K_M. A standard-library extraction program independently replays the fixture assertions, rejects profile and hash inconsistencies, and derives all tables and figures from raw artifacts. It reproduces 130/140, 130/140, and 136/140 passing assertions. Only Q01 and Q03 exercise normal synthesis: both smaller profiles time out on both cases, while 3B publishes Q01 and rejects a schema-valid Q03 draft at the semantic gate. Retrieval expectations remain unchanged across arms. These observations are descriptive single-run evidence, not estimates of population performance. The contribution is therefore not a claim that a model size "wins," but an auditable method for separating retrieval correctness, wire-schema validity, semantic strictness, latency, and failure ownership in a local execution path.

1 Introduction

RAG combines a generator's parametric state with an explicit retrieved memory [7]. That separation is useful for mutable operational records, where a user should be able to inspect the log fragments behind a finding. It is insufficient, however, to conclude that a generated claim is supported: a retriever may return the right records while generation times out, emits invalid structure, links the wrong record, or states a claim not entailed by the returned text. Citation-oriented evaluation similarly distinguishes answer quality from citation completeness and correctness [4].

This paper asks a narrower systems question: *can a local game-log RAG slice make each boundary observable and assign failures without silently crossing the evidence boundary?* The target is not open-domain question answering or production-scale search. It is an evidence-bounded path for a fixed game-operations corpus in which retrieval, synthesis, freshness, typed failure states, and provenance can be inspected separately.

FunQA-Log follows one path: JSONL records, CocoIndex state, PostgreSQL/pgvector retrieval, local Ollama synthesis, a FastAPI stream, and a same-origin web proxy. The design excludes the application's existing Genkit path, cached answers, model-prior answers, and non-CocoIndex evidence as fallbacks. The key systems choice is to make an unsuccessful stage a first-class result rather than an invitation to switch owners. A retrieval outage therefore remains `retrieval_unavailable`; a post-retrieval model outage remains `synthesis_unavailable` while preserving the evidence already found; a schema-valid but semantically unsupported draft becomes `weak_support` and publishes no finding.

The evaluation is intentionally bounded. The frozen corpus contains nine synthetic records, of which eight are indexed; the query manifest contains ten cases. Only Q01 and Q03 traverse normal synthesis. Each of three exact model profiles has one ten-query run. These facts are emitted in `results/run_summary.csv`; the shared corpus and query hashes and each raw run directory appear in the same rows. Consequently, we use raw observations and descriptive comparisons only: no confidence interval, p-value, percentile, or significance test is meaningful here.

This paper makes three contributions:

1. We specify an evidence-bounded orchestration pattern that separates retrieval correctness, response-schema validity, deterministic semantic support, freshness, latency, and owner-aware failure states.
2. We provide an auditable, standard-library artifact builder that verifies common corpus/query hashes, exact model profiles, Q4_K_M quantization, stream identity, provenance membership, and independently recomputed assertions before generating tables and

vector figures.
3. We report a nine-record, ten-query vertical-slice case study and an isolated incremental-dataflow observation. The study exposes both positive behavior (stable retrieval and one strictly accepted 3B finding) and negative behavior (four synthesis timeouts and one semantic rejection) without converting failures into model answers.

2 Related Work

2.1 Retrieval and adaptive correction

Foundational RAG retrieves non-parametric memory for generation and reports gains on the evaluated Wikipedia-backed tasks [7]; it does not establish source faithfulness for local game logs. FLARE performs active retrieval during generation when predicted text contains low-confidence tokens [6]. Self-RAG trains reflection tokens to decide when to retrieve and to critique relevance, support, and utility [1]. CRAG evaluates retrieved evidence and applies corrective actions, including a web-search path [17]. These methods motivate conditional retrieval and correction, but their training regimes, corpora, and external-source assumptions differ from our fixed local evidence boundary. In particular, CRAG's web extension is out of contract here.

Our orchestrator does not learn when to retrieve and does not repeatedly query during generation. It instead freezes one returned set, exposes its hash, and either accepts the generated claims against that set or declines to publish them. This is a systems-control contribution rather than a new retrieval or language-model algorithm.

2.2 RAG evaluation and structured generation

RAGAs decomposes evaluation into faithfulness, answer relevance, and context relevance [3]. ARES uses synthetic data, trained judges, and a human-labeled calibration set to evaluate context relevance, faithfulness, and answer relevance [14]. These frameworks are valuable at larger study scale, but neither is annotation-free ground truth for this bilingual-capable, synthetic game-log fixture. We therefore use the frozen query oracle and deterministic predicates rather than an LLM judge.

Grammar-constrained decoding can guarantee conformity to a formal structure for the evaluated structured NLP tasks [5]; Ollama documents a JSON-Schema `format` interface and recommends validating returned content client-side [10]. Structural conformity does not imply factual support. Our measurement consequently records terminal wire validity, synthesis-schema status, and semantic strictness as separate fields.

2.3 Incremental and local data systems

CocoIndex documents a declarative state-to-target execution model and memoized per-item processing [2]. Differential Dataflow provides a research analogue for propagating changes through declarative computations while reusing prior state [9]; we do not claim that CocoIndex implements Differential Dataflow. pgvector supplies exact nearest-neighbor search by default and optional approximate indexes such as HNSW [12, 8]. Sentence-BERT established efficient independent sentence embeddings for cosine comparison [13]; the current Sentence Transformers API exposes that usage [16]. None of these sources transfers accuracy or latency numbers to this corpus.

Local and edge RAG systems have explored mobile memory, latency, and online-index tradeoffs [11, 15]. Their author-reported preprint results concern different devices and index architectures. Here, "local" denotes the configured execution boundary—local files, PostgreSQL, and a loopback Ollama endpoint in the captured runs—not a privacy, security, energy, or deployment guarantee.

3 System Design

3.1 One evidence path

The frozen architecture contract defines the following path:

$$\text{JSONL} \to \text{CocoIndex} \to \text{PostgreSQL/pgvector} \to \text{retrieval} \to \text{local Ollama} \to \text{typed stream}.$$

CocoIndex discovers local JSONL files. A memoized per-file function validates one record, excludes the designated freshness fixture, computes the excerpt hash, embeds the excerpt, and declares a row keyed by evidence ID. The target contains provenance fields alongside the vector. SQL retrieval filters by snapshot, time, source, project, and entity, then orders by cosine distance with deterministic time and evidence-ID tie breakers. The score is $1 - \text{distance}$ and is never relabeled as a probability.

The returned evidence snapshot is immutable within a dispatch. Each item repeats the query and correlation identities, source and snapshot identifiers, excerpt offsets, trust class, and content hash. A SHA-256 digest over ordered evidence-ID/hash pairs names the set. The artifact builder rejects evidence outside the eight-ID index manifest, malformed content hashes, or identity/snapshot drift.

3.2 Ordered orchestration and typed ownership

A dispatch emits an acceptance frame, retrieval and ranking stages, zero or one evidence snapshot, an optional synthesis stage, and exactly one terminal or cancelled frame. Freshness is checked before synthesis. Empty evidence

Table 1: Terminal ownership is a type-level contract, not a post-hoc label.

Outcome	Owner	Published finding
`supported`	none	strict finding
`no_hits`	none	none
`weak_support`	none	none
`stale_index`	retrieval	none
`retrieval_unavailable`	retrieval	none
`synthesis_unavailable`	synthesis	none

maps to `no_hits`; a request beyond snapshot coverage maps to `stale_index`; deterministic insufficient-evidence cases map to `weak_support`. The terminal mapping fixes outcome, owner, confidence label, and recovery action together, as summarized in Table 1.

This mapping prevents category collapse. For example, a model timeout after retrieval does not invalidate already returned evidence and does not become a retrieval failure. Conversely, a retrieval fault cannot be hidden by calling another generator. Cancellation is outside the six terminal outcomes and preserves any evidence already emitted.

3.3 Schema constraint and semantic gate

The local synthesizer receives the query, visible scope, and only the frozen returned evidence. The system prompt treats excerpts as data, denies tools and alternate sources, and requests three top-level objects: a summary, material claims, and claim-to-evidence links. Ollama receives the Pydantic-derived JSON Schema as its `format`; temperature is zero. The service then parses JSON and validates the draft against that schema.

A second, deterministic gate operates after schema validation. Claim IDs must be contiguous; every link must target returned evidence; untrusted evidence must use the `untrusted_data` relation; every material claim must have returned `supports` evidence with lexical overlap; impossible numeric direction is rejected; and coverage must be complete. Correction semantics are generic: evidence labeled `supersedes` must also support the claim, while the same claim must link prior context or contradiction. The production gate contains no P42, incident-184, or E001/E004/E006/E009 branch; fixture-specific expected values and IDs remain in the runner oracle. A schema-valid draft that fails any semantic predicate produces `weak_support` and no finding. JSON parse or schema failure instead produces `synthesis_unavailable` with reason `malformed_synthesis`. A timeout remains a timeout; it is not counted as either schema-valid or schema-invalid because no draft was observed.

3.4 Incremental flow

The index function is memoized per file and mounted over a live local-file source. The isolated evidence packet contains a baseline, a no-op execution, and a one-change execution. These artifacts are not interleaved with the model runs and their modified E001 row is not compared to the frozen query-run row. This separation avoids treating a dataflow smoke test as an end-to-end retrieval benchmark.

4 Threat Model

The protected properties are evidence membership, provenance identity, owner-correct terminal states, and refusal to publish unsupported claims. Trusted components are the checked-out code, fixture manifests, local database, configured Ollama endpoint, and host operating environment for the observed runs. The corpus itself can contain untrusted text: E009 is an injection canary and is typed `untrusted_data`. Component outages, timeouts, malformed model output, stale snapshots, wrong evidence links, and unsupported claims are in scope.

The design mitigates these threats by constraining inputs, retaining content and set hashes, using parameterized SQL, prohibiting model tools, validating schemas client-side, checking all evidence links against the frozen set, and mapping failures to owners. The fixture runner also records requests made by its own `httpx` client and scans serialized streams for the canary and non-index IDs.

Several threats are explicitly out of scope. A compromised host, database, Ollama binary, model weights, or artifact builder can falsify local evidence. The runner hook does not observe unrelated browser or operating-system traffic. It has no packet capture, HAR, remote attestation, side-channel analysis, or authorization audit. Two reported canary fields—cached-answer use and prior-knowledge answers—are constants in the runner rather than independent detectors. Therefore the negative control supports only the instrumented runner path; it does not prove global network isolation, privacy, confidentiality, or absence of model memorization.

5 Experimental Protocol

5.1 Frozen inputs and arms

The fixture corpus is `sim-game-logs-v1`: nine JSONL records, with E001–E006 and E008–E009 indexed and E007 intentionally excluded. The index snapshot is `sim-index-v1`. The corpus SHA-256 is

`ddf326aa636543a4c77dcf1262bd55e2b388ef1cae709f75f9a6b66f1d8f9ad2.`

The ten-query manifest SHA-256 is

`5fe747e2cf2c8f52cafc2468426e00f41295dfefe9eacbf66091369df35eed33.`

Both values, record/query counts, and each raw directory are emitted in `results/run_summary.csv`.

The compared synthesis profiles are exactly `qwen2.5:0.5b`, `qwen2.5:1.5b`, and `qwen2.5:3b`. Each

fixture manifest, environment allowlist, and synthesis terminal records `Q4_K_M`. Other common fields—CocoIndex version, embedding model, ranking seed, clock, snapshot membership, build ID, and hashes—must match byte-for-byte or structurally before the script writes results. There is one ten-query run per arm. Hardware metadata was not emitted in these raw artifacts and remains unreported. The raw model runs predate the generic semantic-gate change described in Section 3.3. Their quantitative rows remain a historical observation of the frozen run artifacts; the builder re-derives assertions from those artifacts and does not re-execute the current synthesizer. Consequently, the model-arm results below must not be interpreted as an evaluation of the generalized gate.

5.2 Queries and controls

Q01 asks for an exact cooldown change and rationale; Q03 asks for an incident cause and fix while preserving a retraction/correction relationship. These are the only normal synthesis-dependent cases. Q02, Q08, and Q09 are deterministic weak-support boundaries; Q04 is no-hit; Q05 and Q10 inject retrieval unavailability; Q06 injects synthesis unavailability after retrieval; and Q07 requests coverage beyond the frozen snapshot. The all-ten aggregate is therefore dominated by deterministic boundary and fault behavior and is not a model-quality score.

Q10 is the no-fallback negative control. We report the five fields written by each `canary-scan.txt`, but distinguish the three observed scans (runner-hook Genkit URLs, canary occurrences, non-index evidence IDs) from the two runner constants (cache and prior-knowledge counters).

The CocoIndex experiment is a separate three-step execution over nine files: baseline, no-op, and one-change. Its console statistics and displayed elapsed values are parsed from four text artifacts. The experiment has no repetitions and is used to check change selectivity, not to estimate throughput.

5.3 Derived metrics and rejection rules

The builder reconstructs each of fourteen assertions per query from streams and the frozen oracle: outcome, owner, query/correlation identity, minimum evidence count, required top-five IDs, expected rank one, forbidden values, required finding values, scope, scope delta, snapshot, boundary reason, and recovery action. It then requires exact agreement with the raw `results.json` rows. It separately checks the terminal wire shape and strict supported-finding invariants.

For Q01/Q03, synthesis schema status is *valid* when a structured draft reaches semantic validation, *invalid* only for `malformed_synthesis`, and *not observed* when synthesis fails before a draft is available. Semantic rejection is counted only for a schema-valid draft that produces `weak_support`. This prevents a timeout from being mislabeled as a schema defect.

Latency is the runner's per-case monotonic duration from `correlated-spans.json`. The fixture constructs a new sentence-transformer embedder for each case, so the duration includes setup/cache effects and cannot isolate retrieval from synthesis. We show raw Q01/Q03 observations and no percentiles. The full-run duration is retained in the CSV but is not used as query latency.

Table 2: Single-run descriptive results. "Strict" is supported Q01/Q03. "Syn. schema" is valid/invalid/unobserved over those two cases.

Profile	Assert.	Outcome	Strict	Syn. schema
0.5B	130/140	8/10	0/2	0/0/2
1.5B	130/140	8/10	0/2	0/0/2
3B	136/140	9/10	1/2	2/0/0

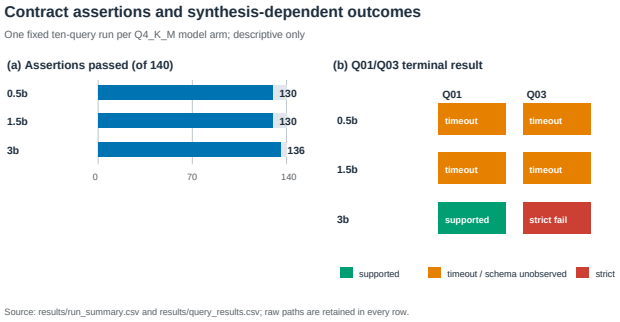

Figure 1: The aggregate assertion difference is explained by two synthesis-dependent cases, not by a broad ten-query model-quality comparison. Source: `results/run_summary.csv` and `results/query_results.csv`.

6 Results

6.1 Contract outcomes

Table 2 and Figure 1 report the independently reconstructed assertions. The 0.5B and 1.5B runs each pass 130/140; the 3B run passes 136/140. All three raw runner exit codes are 1 because at least one assertion fails. Exact-outcome accuracy is 8/10, 8/10, and 9/10, respectively. These counts are rows in `results/run_summary.csv`; all 420 individual comparisons, including expected and observed JSON values, are in `results/assertions.csv`.

The terminal wire-schema validator reports zero failures in each arm. This statement concerns terminal envelope invariants, not the model draft. On Q01/Q03, the smaller profiles produce four timeouts in total, so schema validity is unobserved for those four attempts. The 3B arm returns two schema-valid drafts: Q01 passes the semantic gate,

Table 3: Observed Q01/Q03 terminal results and whole-case latency. One observation exists per cell.

Profile	Case	Outcome	Seconds
0.5B	Q01	synthesis timeout	33.426
0.5B	Q03	synthesis timeout	23.918
1.5B	Q01	synthesis timeout	26.754
1.5B	Q03	synthesis timeout	11.695
3B	Q01	supported	18.465
3B	Q03	strict weak support	19.355

while Q03 is rejected by strict support. No observed Q01/Q03 attempt is classified as malformed synthesis. Thus "zero schema-invalid drafts" must not be read as "all drafts were schema-valid"; four drafts never arrived.

6.2 Retrieval invariants

All three arms retrieve the same ordered evidence IDs for each query that has evidence. Q01 returns E001, E002, E003; Q03 returns E004, E006, E005, preserving the correction directly after the initial hypothesis; Q08 returns only the untrusted E009. Across each arm, the frozen required-ID numerator is 9/9 at rank five, and all ten `required_top_five` assertions pass. The runner's ten `expected_rank_one` assertions also pass per arm, although only four cases define a non-null expected rank-one ID. Per-case numerators, denominators, ordered IDs, exact-ID ranks, raw stream paths, and snapshot IDs are in `results/retrieval_by_query.csv`.

These observations establish contract conformance on this fixture, not general retrieval quality. The corpus and queries were co-designed, the retrieval implementation includes explicit E004/E006 adjacency, and there are no graded judgments for nDCG. We therefore do not report nDCG or transfer any SBERT/pgvector benchmark result.

6.3 Synthesis-dependent cases and latency

Table 3 shows Q01 and Q03. The 0.5B arm times out after 33.426 s on Q01 and 23.918 s on Q03. The 1.5B arm times out after 26.754 s and 11.695 s. The 3B arm supports Q01 after 18.465 s and returns a schema-valid but semantically rejected Q03 draft after 19.355 s. Values are rounded renderings of the millisecond rows in `results/query_results.csv`; Figure 2 plots the same rows without error bars.

The data do not support a monotonic latency claim: Q03 is faster for 1.5B than 3B in these single observations, while Q01 decreases with profile size. Nor do they support a general quality ranking. The only defensible descriptive statement is that 3B is the sole arm to publish a strict finding, on Q01, in this run; all models fail the Q03 expected outcome, for different reasons.

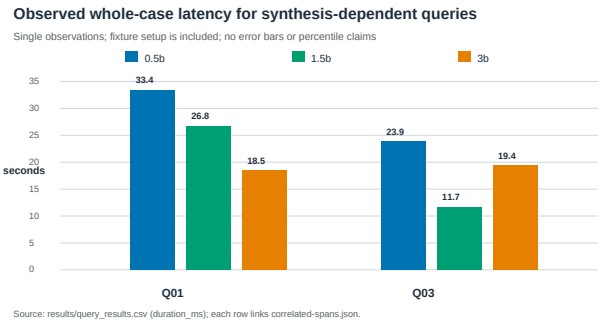

Figure 2: Raw whole-case latency for the two normal synthesis cases. Setup and cache effects are included, and there is one observation per profile/case. Source: `results/query_results.csv`.

Table 4: Isolated incremental-flow observations over nine files.

Phase	Added	Reproc.	Unchanged	Shown s
baseline	9	0	0	12.5
no-op	0	0	9	0.0
one-change	0	1	8	11.4

6.4 No-fallback negative control

For each arm, `canary-scan.txt` reports zero runner-hook Genkit network spans, zero canary occurrences, and zero non-CocoIndex evidence IDs. Q10 remains a typed retrieval-owned unavailable result in `results/query_results.csv`. The same files report zero cached-answer and prior-knowledge use, but the implementation initializes those two counters to zero; they are not active monitors. All five fields and their instrumentation scope are retained in `results/canary_summary.csv`. We therefore describe this as a negative observation for the instrumented fixture path, not proof of system-wide no-fallback behavior.

6.5 Incremental dataflow

Figure 3 and Table 4 show the isolated CocoIndex experiment. Baseline reports nine added files; the no-op reports nine unchanged; one-change reports one reprocessed and eight unchanged. The displayed elapsed observations are 12.5, 0.0, and 11.4 s. They are single console observations and not a benchmark. The useful result is selectivity: the no-op and one-change statistics distinguish unchanged from reprocessed per-file work. `results/cocoindex_incremental.csv` retains each source text path; `results/cocoindex_target_row.csv` retains the post-change E001 row and hash.

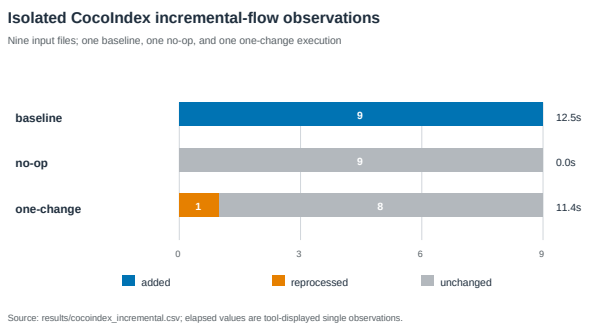

Figure 3: CocoIndex console classifications for baseline, no-op, and one-change executions. Source: `results/cocoindex_incremental.csv`.

7 Failure Analysis

Timeout dominates the smaller arms. Q01 and Q03 each lose five assertions in the 0.5B and 1.5B runs: expected outcome, owner, required finding values, boundary reason, and recovery action. Retrieval assertions still pass. The observed terminal is synthesis-owned `synthesis_unavailable` with `synthesis_timeout`; no draft exists to classify structurally or semantically. The orchestrator therefore preserves evidence and chooses `open_raw_evidence`, as designed, while the fixture oracle expected a supported finding and `inspect_claim_traces`.

3B Q03 is a semantic, not structural, failure. The 3B Q03 attempt reaches the JSON/schema parser and records model token counts, but the deterministic validator returns no finding. The terminal is `weak_support` with owner `none`. Four oracle assertions fail: outcome, required finding values, boundary reason, and recovery action. Owner correctness does not fail because both the expected supported state and observed weak-support state are non-failure outcomes owned by `none`. This distinction is visible only because schema validity and semantic strictness are separate columns.

Aggregate counts conceal causal structure. The 136/140 figure for 3B looks close to the 130/140 figures for smaller models, but the six-assertion difference combines one successful Q01 transition with a differently typed Q03 failure. Eight other cases are deterministic or fault-driven and contribute no normal synthesis-quality comparison. Reporting only the aggregate would therefore overstate both coverage and model evidence.

Failure ownership remains useful when correctness fails. The unsuccessful runner exit codes are not a systems collapse. In the smaller arms, synthesis timeouts are correctly owned and evidence remains inspectable. In 3B Q03, semantic insufficiency is represented without assigning a component outage. Typed failure ownership does not make the answer correct; it makes the failure diagnosable and prevents an alternate source from masquerading as recovery.

8 Limitations and Ethics

External validity. This is a vertical-slice case study with nine synthetic records, eight indexed records, ten hand-authored English queries, one run per profile, and only two normal synthesis cases. It cannot establish production-scale throughput, open-domain retrieval quality, robustness across games, bilingual retrieval effectiveness, user utility, or general model-size trends. The embedding model can process multilingual text, but no Korean query fixture is present.

Statistical validity. There are no repetitions. We report no confidence intervals, p-values, standard errors, percentiles, or significance claims. Query durations include per-case embedder construction and unknown cache/thermal effects. Hardware, operating-system, Ollama-version, model digest, CPU/GPU time, and peak memory were not emitted by these artifacts. Parameter count and latency are therefore confounded with uncontrolled run conditions.

Construct validity. The fixed query oracle checks contract conformance, not unconstrained factuality. Q01/Q03 expected values remain fixture-runner assertions, while the current production semantic gate uses generic returned-evidence coverage, relation, lexical-overlap, numeric-direction, and correction rules. The frozen runs predate that generalization, so they do not measure its effect. Q03 retrieval still explicitly places a correction next to the initial hypothesis. Schema validity only states that a draft reached semantic validation; the conservative semantic predicates are not entailment proofs, and rejection can discard a useful answer. No human evaluation was conducted.

No-fallback and privacy boundaries. The runner observes its own `httpx` requests and serialized streams. It does not observe the full browser, server process tree, or operating system. Two scan fields are constants. "Local execution" names the configured path, not a verified privacy guarantee. The artifact includes synthetic log excerpts and stable fixture identifiers; using the design with real operational logs would require access control, retention, redaction, audit, and incident-response analysis not evaluated here.

Responsible claims. A failed synthesis is not rescued by model prior knowledge. This reduces the chance of presenting untraceable content, but it can reduce availability.

Users must be shown evidence and boundary reasons, not only a supported/unsupported badge. The study makes no claim of acceptance readiness, production deployment, security certification, or replacement of human incident judgment.

9 Artifact Availability

All manuscript artifacts live under `study/genai-game-log-rag/`. From the repository root, regenerate results and figures with:

```
python3study/genai-game-log-rag/scripts/
build_results.py
```

The script uses only the Python standard library. It exits with status 2 before writing if the evidence contract is inconsistent. CSV and figure manifests record SHA-256 digests of generated outputs.

Compile the anonymous two-column paper from its directory with:

```
pdflatex -interaction=nonstopmode
-halt-on-error paper.tex
bibtex paper
pdflatex -interaction=nonstopmode
-halt-on-error paper.tex
pdflatex -interaction=nonstopmode
-halt-on-error paper.tex
```

`REPRODUCIBILITY.md` lists the same commands, artifact map, dependency boundary, and expected extraction summary.

10 Conclusion

An evidence path is more useful when its failure modes are as explicit as its successful findings. FunQA-Log combines incremental local indexing, deterministic retrieval provenance, constrained local synthesis, strict claim–evidence validation, and typed ownership without an alternate fallback. In this limited case study, retrieval contracts remain stable while synthesis behavior differs: two smaller profiles time out on both normal synthesis cases, and 3B accepts one case while rejecting the other semantically. The result is not a general ranking of Qwen models. It is a reproducible demonstration that schema validity, semantic support, latency, and owner-correct failure can be measured separately rather than compressed into one RAG score.

A Raw Artifact Map

The authoritative model-run directories are:

- `_workspace/current/qa/evidence/stage-1/fixture-run-qwen0_5b-schema/`
- `_workspace/current/qa/evidence/stage-1/fixture-run-qwen1_5b/`
- `_workspace/current/qa/evidence/stage-1/fixture-run-qwen3b/`

Each contains `fixture-manifest.json`, `index-manifest.json`, `queries.json`, `results.json`, `streams.json`, `correlated-spans.json`, `canary-scan.txt`, copied hash files, command/environment records, timestamps, duration, exit code, stdout, and stderr.

The isolated incremental artifacts are:

- `_workspace/current/engineering/evidence/stage-1/cocoindex-experiment-baseline.txt`
- `_workspace/current/engineering/evidence/stage-1/cocoindex-experiment-noop.txt`
- `_workspace/current/engineering/evidence/stage-1/cocoindex-experiment-one-change.txt`
- `_workspace/current/engineering/evidence/stage-1/cocoindex-experiment-target-row.txt`

B Derived Artifact Schema

`results/run_summary.csv` is one row per model arm. `results/query_results.csv` is one row per model/query pair and contains identities, expected and observed states, duration, ordered evidence IDs, schema/semantic status, token fields, and raw paths. `results/assertions.csv` contains all independently reconstructed assertions. `results/retrieval_by_query.csv` contains retrieval numerators, denominators, ranks, and raw stream paths. `results/canary_summary.csv` states instrumentation scope. The two CocoIndex CSV files preserve execution classes and the isolated post-change target row.

C Rejection Tests

Before aggregation, the builder requires: exact profile-to-directory mapping; Q4_K_M in both manifest and environment; identical common profile fields; query bytes matching the query hash; the current corpus matching the corpus hash; ten unique cases; ten spans; fourteen known assertions per case; independently reconstructed assertions equal to raw rows; terminal identities equal to query identities; evidence IDs inside the index manifest; lowercase 64-hex content hashes; snapshot consistency; finding links confined to returned evidence; and exit code consistent with assertion failure. These checks make a copied or edited artifact fail closed rather than silently entering a figure.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

[2] CocoIndex Project. Core concepts. Official CocoIndex v1 documentation, 2026. Accessed 2026-08-11.

[3] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024.

[4] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[5] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[6] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[8] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[9] Frank McSherry, Derek G. Murray, Rebecca Isaacs, and Michael Isard. Differential dataflow. In *Proceedings of the 6th Biennial Conference on Innovative Data Systems Research*, 2013.

[10] Ollama Project. Structured outputs. Official documentation, 2026. Accessed 2026-08-11.

[11] Taehwan Park, Geonho Lee, and Min-Soo Kim. MobileRAG: A fast, memory-efficient, and energy-efficient method for on-device RAG. *arXiv preprint arXiv:2507.01079*, 2025.

[12] pgvector Project. pgvector: Open-source vector similarity search for PostgreSQL. Official repository and reference documentation, 2026. Accessed 2026-08-11.

[13] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019.

[14] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2024.

[15] Korakit Seemakhupt, Sihang Liu, and Samira Khan. EdgeRAG: Online-indexed RAG for edge devices. *arXiv preprint arXiv:2412.21023*, 2024.

[16] Sentence Transformers Project. Semantic textual similarity. Official Sentence Transformers documentation, 2026. Accessed 2026-08-11.

[17] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.